

pair 정리

두 개의 값을 하나로 묶는 간단한 표준 타입입니다.

사용처 (Where)

- 번호와 이름, 좌표 x/y처럼 두 값이 항상 같이 움직일 때
- 정렬 기준을 두 개 값으로 잡을 때
- map의 원소를 이해할 때

방법 (How to Use)

- pair<첫자료형, 둘째자료형>로 선언
- first, second로 접근
- make_pair 또는 중괄호로 생성
- vector<pair<int,int>>처럼 컨테이너와 함께 사용

기본 정보

- 필요 헤더: #include <utility>
- 기본 선언: pair<int, string> p = {1, "one"};

왜 써야 할까? (Why)

- 간단한 두 값 묶음에 구조체보다 빠르게 작성 가능
- 정렬과 map에서 자연스럽게 사용됨
- 반환값 두 개를 묶어 전달하기 좋음

주요 함수

함수/문법	의미
first	첫 번째 값
second	두 번째 값
make_pair(a,b)	pair 생성
비교 연산	first부터 비교 후 second 비교

기본 코드 예시

```
#include <iostream>
#include <utility>
using namespace std;

int main() {
    pair<int, string> student = {1, "Kim"};
    cout << student.first << '\n';
    cout << student.second << '\n';
}
```

pair 비교와 응용

STL을 직접 구현이나 C 스타일 코드와 비교하고, 실제 문제에서 쓰는 방식을 확인합니다.

시간 복잡도

동작	복잡도
생성/접근	$O(1)$
비교	각 값 비교 비용
vector<pair> 정렬	$O(n \log n)$

STL vs 직접 구현 vs C 스타일

구분	STL	직접 구현	C 스타일
표현력	의도를 타입으로 직접 표현	구조체/클래스를 직접 작성	배열/struct 수동 관리
코드 길이	짧고 표준적인 형태	필드와 생성자 작성 필요	초기화와 접근 실수 가능
유지보수	다른 STL과 자연스럽게 연동	설계가 늘어날수록 관리 필요	가독성이 떨어지기 쉬움
추천 상황	간단한 값 묶음/조건 전달	도메인 의미가 큰 객체	C API와 맞춰야 할 때

응용: 점수와 이름을 함께 정렬

점수와 이름을 pair로 묶어 vector에 저장한 뒤 점수 기준으로 정렬합니다.

```
#include <iostream>
#include <vector>
#include <algorithm>
#include <utility>
using namespace std;

int main() {
    vector<pair<int, string>> v = {{90, "Kim"}, {80, "Lee"}, {90, "Park"}};
    sort(v.begin(), v.end());

    for (auto p : v) {
        cout << p.first << ' ' << p.second << '\n';
    }
}
```

처음 배울 때 자주 하는 실수

- first/second 이름만으로 의미가 불분명하면 구조체가 더 좋음
- 정렬은 기본적으로 first 우선임
- 세 값 이상이면 tuple 또는 struct 고려

비슷한 항목과 차이

- tuple은 세 개 이상 값을 묶을 수 있음
- struct는 필드 이름을 줄 수 있어 의미가 명확
- map의 원소는 `pair<const Key, Value>` 형태

한 줄 정리: `pair`은(는) '번호와 이름, 좌표 x/y 처럼 두 값이 항상 같이 움직일 때' 상황에서 먼저 떠올리면 좋은 STL입니다.